

Note: This material has been adapted from a project by Yannik Scholz.

This assignment is concerned with the implementation of a *Pick and Place* system for the myCobot 280 robot arm.

The following exercises are solved with Python and ROS 2 Jazzy. The lab combines a vision-based object detection node, custom ROS 2 messages, and a motion node for autonomous pick-and-place execution with the myCobot 280.

Before the assignment, read this document entirely and review the code structure of the provided ROS 2 packages. Make sure you understand the data flow from camera detection to robot motion execution.

Learning Objectives: Pick and Place Lab

In this lab, students investigate and extend a vision-guided pick-and-place system for the myCobot 280. They first run the existing application, use its text-based control menu to sort and stack cubes, and trace how camera images lead to robot motions. They then improve the system's color detection, camera-to-robot calibration, object recognition, and user interface.

By the end of the lab, students will be able to:

- Explain how the camera, vision, Brain, MoveIt, and controller nodes interact through ROS 2 topics, services, and actions.
 - Operate the existing sorting and stacking sequences, document their behavior, and identify limitations using observations from the physical setup.
 - Analyze HSV-based color segmentation and calibrate color thresholds from pixels selected within an object region.
 - Improve binary masks and contour filtering to make object detection more robust under changing lighting conditions.
 - Detect two ArUco markers with known poses, estimate the camera extrinsic parameters, and check the estimate against the known camera height and marker reprojection errors.
 - Use calibrated camera parameters to transform detected image positions into the robot base frame and measure the resulting position error at independent workspace locations.
 - Analyze segmented object regions and classify their color and shape, distinguishing cubes from cylinders.
 - Extend the vision service and the Brain node so that objects can be selected by color and shape, represented appropriately in the MoveIt planning scene, and picked through the control menu.
-

- Evaluate the modified system by comparing detection and pick-and-place performance with the original implementation.

The lab connects image processing and camera calibration with ROS 2 communication and robot manipulation. Students gain practical experience in testing an existing system, identifying its limitations, and validating targeted improvements on the real robot.

Note on task selection. Complete **two** of the following five tasks: camera calibration using ArUco markers, robust HSV masking, calibration of HSV masks from selected image examples, object shape detection, or extension of the object interface. You may choose any two tasks. Document and evaluate the results of both selected tasks; the remaining tasks are optional.

Vision-Guided Grasping and Visual Servoing

Vision-guided grasping uses camera images to locate an object and determine where the robot should place its end effector. A typical system detects the object, estimates a suitable grasp pose, transforms that pose into the robot base frame, and plans an approach and grasp motion. After grasping, the robot lifts the object and transports it to a destination. The quality of each step depends on reliable object detection, camera calibration, and a consistent definition of coordinate frames.

A grasp pose includes more than an object center. It must specify an end effector position and orientation that allow the gripper to contact the object without colliding with its surroundings. For the overhead camera and vacuum pump used in this lab, a simple strategy is to approach the center of the object's visible top surface with the pump pointing downward. The robot first moves to a hover pose, descends toward the surface, activates the pump, and retracts with the object. Errors in the estimated object position can cause the pump to miss the surface, particularly for small objects.

Position-Based Visual Servoing

Visual servoing uses camera measurements to guide robot motion. In *position-based visual servoing* (PBVS), the system estimates the three-dimensional pose of the target from the image and controls the robot using the resulting pose error. For example, if the estimated grasp pose in the robot base frame is ${}^B T_G$ and the current end effector pose is ${}^B T_E$, the controller can use their relative transform

$${}^E T_G = \left({}^B T_E\right)^{-1} {}^B T_G$$

to determine the motion required to bring the end effector to the grasp pose. The superscripts identify the frame in which each pose is expressed.

Obtaining ${}^B T_G$ requires a chain of estimates. The camera detects the object in image coordinates, camera calibration relates image measurements to rays in the camera frame, and the camera extrinsic parameters relate the camera frame to the robot base frame. If the object lies on a surface of known height, the intersection of a camera ray with that surface provides its three-dimensional position. The desired grasp orientation must also be defined in the robot frame. Consequently, errors in intrinsic calibration, camera pose, surface height, or object detection propagate into the commanded grasp pose.

PBVS can operate either with a single measurement or with repeated visual feedback. In a single-measurement approach, the robot plans a motion from one estimated object pose and executes it without updating the target. In closed-loop PBVS, the camera observes the scene again during the approach; the pose error is recomputed and the motion command is updated. Repeated feedback can compensate for some disturbances and initial localization errors, although it requires the target to remain observable and the measurements to arrive in time.

The provided pick-and-place application follows a single-measurement approach: the vision node supplies object coordinates to the Brain node, which then commands planned

robot motions. It does not continuously recalculate the grasp pose while the end effector approaches the object. The calibration and detection exercises in this lab improve the pose estimate on which those motions depend. They also provide a foundation for a future closed-loop visual-servoing controller.

MoveIt

MoveIt 2 is a motion-planning framework for robotic manipulators in ROS 2. It connects a robot model, its current joint state, a representation of the workspace, and motion planners. Given a target pose for the end effector, MoveIt can compute a joint trajectory while checking constraints and collisions. The resulting trajectory is sent to a robot controller for execution.

A central concept is the *planning scene*. It contains the robot's current state and geometric representations of objects in its environment. Objects can be added as collision objects, removed, or attached to a robot link. During pick-and-place, a cube initially belongs to the environment. Once grasped, it is attached to the end effector so that motion planning accounts for the object moving with the robot.

MoveIt supports different types of motion requests. A pose goal specifies where the end effector should arrive, while a Cartesian path specifies waypoints for its motion through space. In this lab, pose goals are used to move between locations in the workspace. Cartesian paths are used for the short downward approach to a cube and the subsequent upward retreat. Planning a path and executing its trajectory are separate steps.

The **Brain** node coordinates MoveIt with the vision and pump-control components. It obtains a detected cube pose from the vision node, updates the planning scene, requests robot motions, and switches the vacuum pump on or off at the appropriate points. MoveIt plans the robot motion; object detection and pump actuation are handled by the other components of the application. Accurate camera-to-robot calibration is therefore essential: a collision-free trajectory to an incorrectly estimated pose can still miss the cube.

System Components and Technical Details

This section describes the hardware and software components used. The overall system uses open-source frameworks and manufacturer software to perform the pick-and-place task successfully.

Table 1 provides a concise overview of the technology used:

myCobot 280pi

The **myCobot 280pi** by Elephant Robotics is the system's main actuator. It is a collaborative robot arm designed specifically for education and research. A distinctive feature of this model is direct control via an integrated Raspberry Pi 4.

The following technical specifications of the cobot are important for accurate pick-and-place operations:

Component	Model / Version	Manufacturer	Documentation
Robot arm	myCobot 280pi	Elephant Robotics	link
Operating system	Ubuntu 24.04 LTS	Canonical	link
Middleware	ROS 2 Jazzy	Open Robotics	link
Language	Python 3.12	Python Foundation	link
Robot API	pymycobot	Elephant Robotics	link
Motion planning	MoveIt 2	PickNik Robotics	link
Image processing	OpenCV 4.6	Open Source	link

Table 1: Overview of the system components

- **Kinematics:** 6 joints
- **Workspace:** maximum reach of 280 mm
- **Precision:** ± 0.5 mm
- **Payload:** 250 g

The setup also includes a flat base supplied by the manufacturer, with recesses for the sorting containers and a camera mount. A USB camera attached to this mount captures a top-down view of the base. In addition, a pump is attached as the end effector at the arm's sixth joint and is used to lift the cubes.

A photograph of the setup is shown in Fig. 1:

Software Architecture Figure 2 shows the ROS 2 software architecture, including the main nodes and interfaces.

Brain

The **Brain node** is the system's central logic unit. Its interfaces with the other nodes are described in detail below.

Camera

The **Camera node** provides the system's camera input. It processes images from the USB camera and publishes them as `sensor_msgs/msg/Image` messages to the `/image` topic at a rate of 10 Hz.

Vision

The **Vision node** provides the system's perception. It subscribes to the `/image` topic and determines the image coordinates of the centers of the cubes' top faces. On request, it sends these coordinates to the **Brain node** () via the `/get_cube_coords` service, which uses the custom type `GetCubeCoords.srv`.

The coordinates are determined as follows:



Figure 1: Basic project setup with robot, base, camera, containers, and cubes

- **HSV masking:** Masks are created for the HSV ranges of the possible colors (red, yellow, green, and blue).
- **Shape detection:** Within these masks, rectangles with side lengths between 50 px and 150 px and an aspect ratio (short side / long side) greater than 0.75 are selected.
- **Selection:** Only one cube of each color is permitted. Therefore, the rectangle with the largest aspect ratio is selected.
- **Coordinate determination:** Finally, a linear transformation maps the pixel coordinates of the selected rectangle's center to coordinates in the robot frame.

The **Vision node** also provides a visual debugging feature. The streamed images are displayed in a window, with the detected rectangles and their centers overlaid. The HSV values and pixel coordinates at the current mouse position are displayed, enabling direct

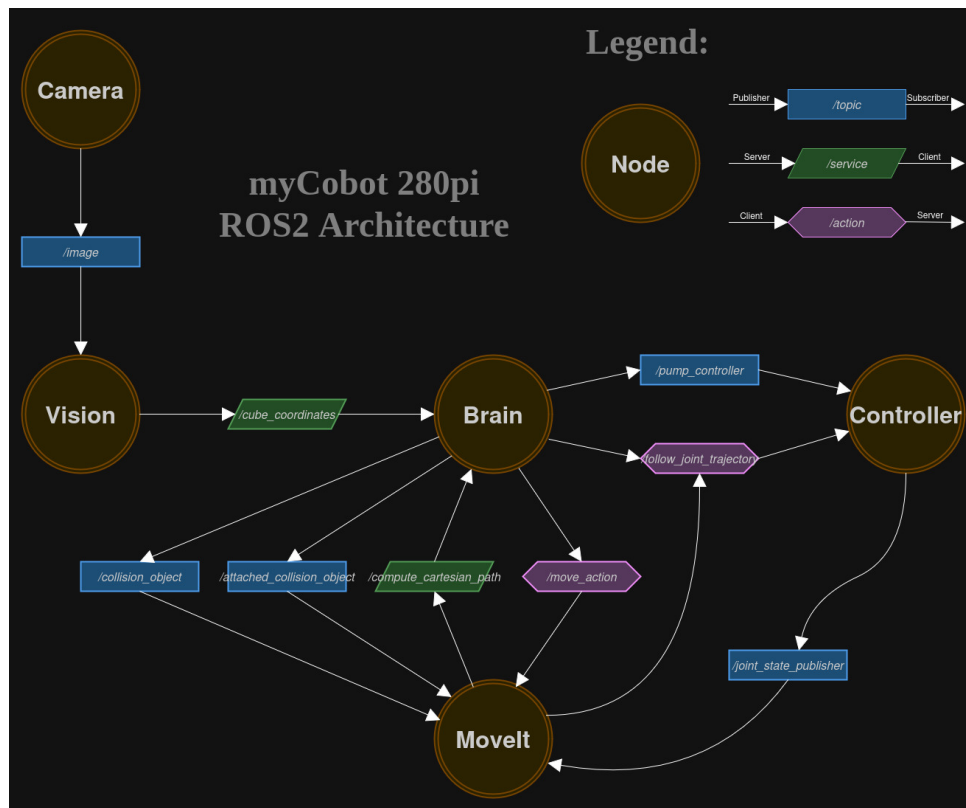


Figure 2: ROS 2 software architecture with nodes, topics, services, and actions

calibration of the HSV ranges for the local lighting conditions.
The debugging window is shown in Fig. 3:

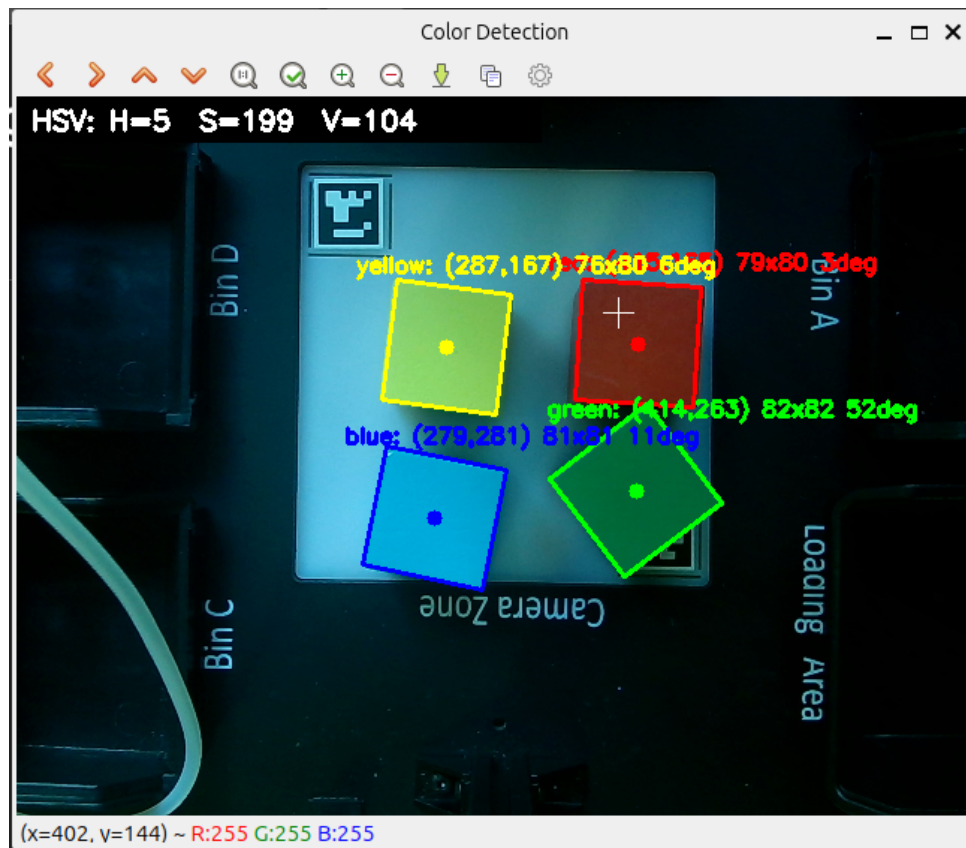


Figure 3: Debugging window with detected cubes and HSV values at the mouse pointer (white crosshair)

MoveIt

The **MoveIt** node handles the system's motion planning. It communicates directly with the **Brain** node () through several interfaces:

Kind	Name	Type
Topic	<i>/collision_object</i>	moveit_msgs/msg /CollisionObject
Topic	<i>/attached_collision_object</i>	moveit_msgs/msg /AttachedCollisionObject
Service	<i>/compute_cartesian_path</i>	moveit_msgs/srv /GetCartesianPath
Action	<i>/move_action</i>	moveit_msgs/action /MoveGroup
Action	<i>/arm_group_controller</i> <i>/follow_joint_trajectory</i>	control_msgs/action /FollowJointTrajectory

Table 2: Interfaces between the MoveIt node and the Brain node

Motion Planning Through the *compute_cartesian_path* service, the **MoveIt node** acts as a server and provides the **Brain node**, acting as a client, with a straight-line trajectory from the current end effector pose to a desired pose along the Cartesian world-coordinate axes. This is particularly useful for moving the end effector down onto an object from above during grasping and then safely retracting with the grasped object.

The */move_action* action allows the **Brain node**, acting as a client, to instruct the **MoveIt node**, acting as a server, to plan and subsequently execute a trajectory to a target pose.

The computed trajectory is executed through another action, */arm_group_controller/follow_joint_trajectory* with the **MoveIt node** as client and the **Controller node** () as server.

Environment Simulation The */collision_object* and */attached_collision_object* topics provide an interface between the **Brain node**, as publisher, and the planning environment in the **MoveIt node**, as subscriber. These topics are used to add and remove collision objects such as cubes in the planning environment and to attach them to and detach them from the pump.

An RViz simulation is available for trajectory simulation. It graphically represents the **myCobot 280pi**, the workspace, and the cubes, including their collision geometries. Trajectories and sequences can therefore be observed without using the physical robot. Because the simulated robot also serves the

/arm_group_controller/follow_joint_trajectory action, the simulation should not be used at the same time as the **Controller node**.

An example is shown in Fig. 4:

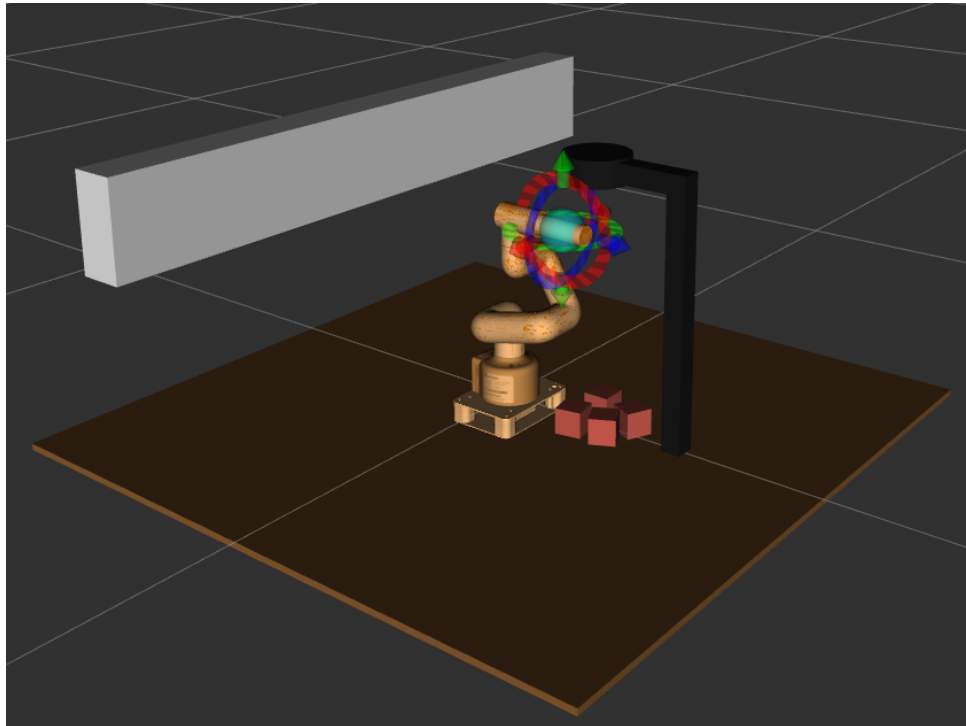


Figure 4: RViz simulation of the IRF workspace with an example cube arrangement

Controller The **Controller node** provides the system's hardware interface. It serves the `/arm_group_controller/follow_joint_trajectory` action, which has two clients: the **Brain node** () and the **MoveIt node** (). The **MoveIt node** requests the execution of free-space trajectories, while the **Brain node** commands the execution of Cartesian trajectories. This arrangement is necessary because MoveIt provides no action for executing Cartesian trajectories; it only provides the service described in to compute them.

Another important topic is `/joint_state_publisher`. The **Controller node** publishes the physical cobot's current joint angles to this topic at 20 Hz. The **MoveIt node** () subscribes to it to determine the current robot configuration at the start of trajectory planning.

The `/pump_controller` topic also acts as an interface between the **Brain node**, as publisher, and the **Controller node**, as subscriber, for targeted control of the pump during grasping.

Main Logic and User Interface The top-level logic is implemented in the **Brain node**. Its code implements the cube-sorting and cube-stacking sequences, as well as the user interface.

Grasping a Cube Grasping a cube follows this sequence:

- The end effector moves to a hover pose 10 cm above the cube's top face, with the pump pointing downward.
- The pump descends along a Cartesian trajectory to the cube's top face.

- The pump is activated.
- The end effector retracts along a Cartesian trajectory to the hover pose.

Placing a Cube in a Container Placing a cube in a specified **container** follows a simple sequence:

- The end effector moves to a hover pose approximately 15 cm above the fixed container position, with the pump pointing downward.
- The pump is deactivated.
- The end effector moves to its fixed home pose.

Stacking the Cubes

Placing a cube on another **cube** follows this sequence:

- The number of cubes in the stack after this operation is passed in, and the additional height required is calculated from it.
- The end effector moves to a hover pose 6 cm above the lower cube's top face, with the pump pointing downward.
- The end effector descends along a Cartesian trajectory until the attached cube rests on the lower cube.
- The pump is deactivated.
- The end effector retracts along a Cartesian trajectory to the hover pose.
- The end effector moves to its fixed home pose.

The **stacking** sequence uses the functions described above:

- If the green cube is detected, it is grasped ().
 - The end effector moves to its fixed home pose.
 - If the green cube was grasped, it is placed on the red cube ().
 - If the green cube was placed on the red cube, the procedure is repeated to place the blue cube on the green cube.
 - If the green cube was **not** placed on the red cube, the procedure is repeated to place the blue cube on the red cube.
-

Figure 5 shows the placement of the blue cube during an example stacking sequence, both in the simulation and at the physical workspace:

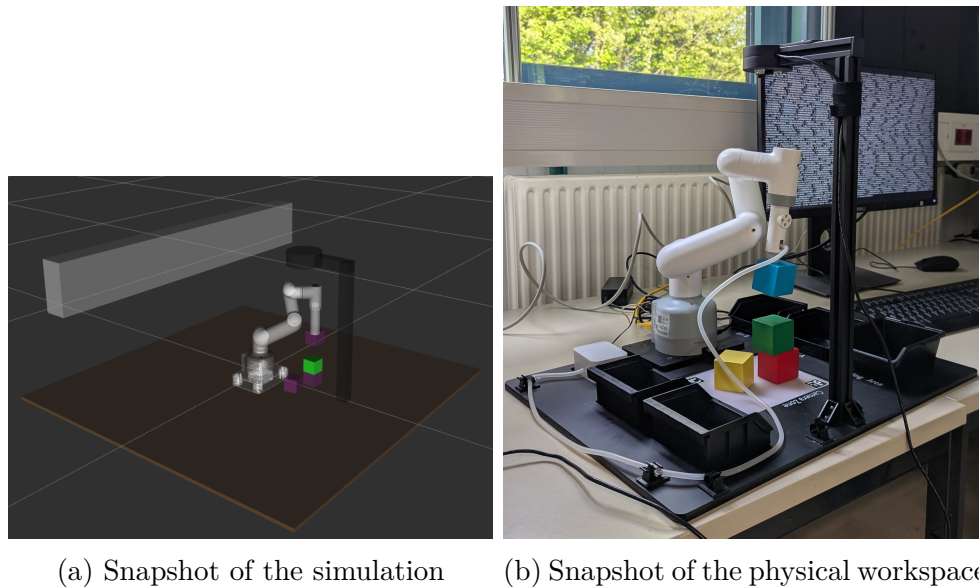


Figure 5: Snapshots during the placement of the blue cube in a stacking sequence

User Interface The user interface opens when the project is launched. It is a simple control menu in Bash, operated through text commands. Its structure is shown in Fig. 6:

```
--- MyCobot Control Menu ---
1 - Stack Cubes: red -> green -> blue
2 - Sort Cubes into Bins
-----
Select mode:
█
```

Figure 6: Project control menu

Selecting a mode starts the corresponding sequence:

- **1:** Stack the cubes
- **2:** Sort the cubes

Selecting mode 1 starts the stacking sequence. It can be interrupted with *Ctrl-C*, although this also closes the control menu. Once the sequence finishes successfully, the user

automatically returns to the top level of the control menu.

In mode 2, the user has full control over sorting. First, the user selects a cube by entering its color. After the cube is grasped, the user selects the destination container. The cube is placed there, and the user is prompted to select another cube. The user can return from mode 2 to the top level of the control menu by entering `exit`, provided that the robot is not moving at that moment.

Assignment: Explore the Existing Pick-and-Place System

Before modifying the provided code, run the existing system and investigate how the robot performs pick-and-place tasks. Use the text-based control menu implemented in `brain.py` and observe both the camera display and the robot's motion.

- 1) **Prepare the setup.** Inspect the robot workspace, camera view, cubes, containers, and vacuum pump. Start the provided ROS 2 nodes according to the lab instructions. Check that the vision display detects the cubes before starting a robot motion. Perform the first motions under supervision and keep the workspace clear.
- 2) **Try the sorting mode.** Select mode 2 in the control menu. Choose a detected cube by color and then select a destination container. Repeat the procedure with at least two cubes and different containers. Observe the approach, grasp, lift, transport, release, and return-to-home motions.
- 3) **Try the stacking mode.** Place the cubes in suitable starting positions and select mode 1. Observe the order in which cubes are picked and stacked. Note what happens when a required cube is not detected or a motion cannot be completed.
- 4) **Trace the information flow.** For one pick-and-place operation, identify which node supplies the camera image, which node determines the cube coordinates, how the `Brain` node requests those coordinates, and which interfaces are used for motion planning, trajectory execution, and pump control.
- 5) **Document your observations.** Record the initial cube positions, menu selections, destination containers, and outcomes. Describe at least two limitations or unexpected behaviors that you observe, such as missed detections, inaccurate grasp positions, failed trajectories, or restrictions of the current menu.

Deliverable. Submit a short report with your observations and a diagram or ordered list showing the data flow from camera image to completed pick-and-place motion. Use these observations as a baseline when evaluating your subsequent improvements to calibration, object detection, and the user interface.

Recalibrating the Camera-to-Robot Transformation

Existing implementation. In `vision.py`, the center (c_x, c_y) of a detected cube is converted to robot coordinates in `send_cube_coords()` using fixed offsets and scale factors:

$$\begin{aligned}x_B &= 0.08 + \frac{c_y - 55}{280} 0.15, \\y_B &= -0.075 + \frac{c_x - 185}{280} 0.15.\end{aligned}$$

The service also returns a fixed height of $z_B = 0.01\text{ m}$ and converts the image angle returned by `cv2.minAreaRect()` directly to radians. This mapping was determined for the previous camera configuration. Since the camera intrinsics and its pose have changed, the constants can no longer be assumed to produce accurate robot coordinates.

Calibration setup. Two ArUco markers are fixed at known positions and orientations in the robot base frame B . Their IDs and side lengths are known. The camera height h above the marker plane is also known. Define the marker plane as $z_B = 0$, or state its actual height if it differs from the robot-base origin.

Task. Replace the fixed mapping with a calibrated transformation:

- 1) **Obtain the new camera intrinsics.** Determine the camera matrix K and distortion coefficients $\mathbf{d}$ for the image resolution used by `vision.py`. Explain how distortion is accounted for when processing detected image coordinates.
- 2) **Define the marker geometry.** Calculate the coordinates of all four corners of each marker in frame B from its known size, position, and orientation. Specify the corner ordering and associate each detected marker with its geometry using its ArUco ID.
- 3) **Estimate the camera extrinsics.** Detect both markers in an image and use their eight corner correspondences, K , and $\mathbf{d}$ to estimate the robot-to-camera transform with `cv2.solvePnP()`:

$$\mathbf{p}_C = R_{CB}\mathbf{p}_B + \mathbf{t}_{CB}.$$

Invert the estimated transform to obtain

$$R_{BC} = R_{CB}^\top, \quad \mathbf{t}_{BC} = -R_{CB}^\top \mathbf{t}_{CB}.$$

Select a pose consistent with the physical camera placement and check that its estimated height $t_{BC,z}$ agrees with the measured height h . State how the node behaves if one of the markers is missing.

- 4) **Transform detected object positions.** Replace the constants in `send_cube_coords()` with a dedicated pixel-to-robot function. For an undistorted image point (u, v) , obtain its camera ray $\mathbf{r}_C$ from K . Intersect the transformed ray

$$\mathbf{p}_B(\lambda) = \mathbf{t}_{BC} + \lambda R_{BC} \mathbf{r}_C$$

with the plane $z_B = z_{\text{object}}$ containing the detected top surface:

$$\lambda = \frac{z_{\text{object}} - t_{BC,z}}{(R_{BC} \mathbf{r}_C)_z}.$$

Specify the relevant surface height in the robot base frame. Do not assume that the top of a cube lies in the marker or tabletop plane. Reject intersections outside the robot workspace or behind the camera.

- 5) **Review the returned pose.** Check whether the fixed z_B value in the current service response is appropriate for the chosen object reference point. Determine whether the angle from `cv2.minAreaRect()` needs to be transformed into the robot frame before it is used as object yaw.
- 6) **Validate the transformation.** Reproject the eight known marker corners and report the mean and maximum reprojection errors in pixels. Place an object at several independently measured workspace positions, including positions near its edges. Compare the returned robot coordinates with the measured coordinates and report the mean and maximum position errors in millimeters. Repeat the extrinsic calibration with several images and assess its consistency.

Deliverables. Submit the updated `vision.py`, the calibration parameters with explicit coordinate-frame and unit conventions, an annotated image showing the detected and reprojected marker corners, and a table of validation errors.

Note. Use the *corners* of both markers, not only their centers. Even with a known camera height, two marker-center correspondences alone do not generally determine the complete camera pose.

Robust HSV-Based Cube Detection

Existing implementation. The `Vision` node subscribes to `camera/image`. In `img_callback()`, it converts each camera image from BGR to HSV and creates a binary mask for each of four cube colors: red, yellow, green, and blue. Red uses two hue intervals because red lies at the boundary of the hue scale. For each mask, the node finds contours and uses `cv2.minAreaRect()` to identify approximately square regions with side lengths between 50 and 150 pixels. It selects the candidate whose aspect ratio is closest to one and stores its center and orientation in `self.detected_cubes`. The mouse-position overlay displays HSV values to help inspect the camera image.

The current masks use fixed thresholds and are passed directly to contour detection. Consequently, changes in lighting, shadows, reflections, and isolated pixels can affect detection. In addition, detections from earlier frames remain in `self.detected_cubes` when a cube is no longer visible.

Task. Improve the HSV masking so that cube detection remains reliable under different lighting conditions and does not report cubes that have disappeared. Make the changes in `vision.py` as follows:

- 1) **Inspect and calibrate the thresholds.** In the `colors` dictionary inside `img_callback()`, review the hue, saturation, and value limits for each color. Use the existing mouse-position HSV overlay to measure representative pixels on cube surfaces and in the background. Note that OpenCV represents hue on the range 0–179 for an 8-bit HSV image; correct any threshold outside that range.
- 2) **Clean each binary mask.** After combining the results of `cv2.inRange()` and before calling `cv2.findContours()`, apply suitable morphological operations, such as opening to remove small isolated regions and closing to fill small gaps. Choose and justify the structuring element and its size. Preserve the approximate shape and center of each cube.
- 3) **Reject unreliable candidates.** Extend the checks in the contour loop, if necessary, so that noise and background objects are less likely to pass the square test. For example, consider contour area, the fraction of the bounding rectangle covered by the contour, and whether the candidate lies within the workspace. Keep the existing size and aspect-ratio checks as a starting point.
- 4) **Prevent stale detections.** Update `self.detected_cubes` for each processed frame so that a color is reported only when a valid cube of that color was detected in the current frame. Check that `send_cube_coords()` returns its existing not detected response when a cube is removed.

Useful OpenCV and NumPy functions. The following functions may help with the implementation:

- `cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`: Converts a camera image from BGR to HSV before color segmentation.
- `cv2.inRange(hsv, lower, upper)`: Creates a binary mask whose white pixels fall within the specified HSV limits.
- `cv2.bitwise_or(mask1, mask2)`: Combines two masks, for example the two hue intervals needed to represent red near the ends of OpenCV's hue range.
- `cv2.getStructuringElement(shape, size)`: Creates a kernel for morphological filtering. Its shape and size determine which small image structures are affected.
- `cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)`: Removes small isolated foreground regions by applying erosion followed by dilation.
- `cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)`: Fills small gaps and holes by applying dilation followed by erosion.
- `cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`: Extracts the outer boundaries of connected regions in a binary mask.

- `cv2.contourArea(contour)`: Measures a candidate region's area in square pixels; useful for rejecting small detections.
- `cv2.minAreaRect(contour)`: Finds a rotated bounding rectangle. Its side lengths can be used for the existing size and aspect-ratio checks.
- `cv2.boundingRect(contour)`: Finds an axis-aligned bounding rectangle. Comparing its area with the contour area provides a possible measure of how much of the rectangle is filled.
- `cv2.pointPolygonTest(workspace, point, False)`: Checks whether a candidate center lies inside a polygon defining the valid workspace.
- `numpy.zeros_like(mask)`: Creates an empty mask with the same dimensions and data type, useful when constructing or displaying intermediate results.

To prevent stale detections, create a new detection dictionary at the start of each successfully processed image and replace `self.detected_cubes` with it after examining all colors. This part is a state-management change in Python rather than an OpenCV operation.

Evaluation. Test the detector with cubes in several positions and orientations, and with at least two lighting conditions. Include cases with shadows or reflections and a frame in which a previously detected cube has been removed. Show the original and improved masks for representative frames. Report any false detections or missed cubes, and explain your threshold and filtering choices. The node must continue to provide cube coordinates through the existing `/cube_coordinates` service.

Assignment: Interactive Calibration of HSV Thresholds

The fixed HSV thresholds in the vision module may fail when lighting or camera settings change. Develop a Python program that estimates suitable thresholds from an object selected in a camera image.

Task. Implement the following steps using OpenCV and NumPy:

- 1) Load a color image and display it in an OpenCV window. Let the user mark the boundary of a colored object by clicking its vertices. Draw the polygon as the user clicks. A right-click removes the last vertex; pressing **Enter** completes the polygon.
 - 2) Create a binary mask of the polygon using `cv2.fillPoly()`. Convert the image from BGR to HSV and extract only the HSV pixels inside the selected region. Exclude a narrow band along the polygon boundary to reduce the influence of background pixels.
-

- 3) Analyze the pixel distribution. Report the median and standard deviation of saturation and value. Determine preliminary thresholds using

$$x_{\min} = \max(0, \text{median}(x) - 2\sigma_x), \quad x_{\max} = \min(255, \text{median}(x) + 2\sigma_x), \quad x \in \{S, V\}.$$

Here, σ_x is the *standard deviation*, not the variance: variance has squared units and cannot meaningfully be added to a pixel value.

- 4) Handle hue separately. In OpenCV, hue ranges from 0 to 179 and wraps around: values near 179 and 0 may describe nearly the same color. Estimate a hue center and spread with this circular range in mind. If the resulting interval crosses 0 or 179, represent it as two ranges whose masks are combined with `cv2.bitwise_or()`.
- 5) Apply the resulting threshold or thresholds to the entire image using `cv2.inRange()`. Display the original image, the selected polygon, the resulting binary mask, and the masked image. Print the threshold values in a form that can be copied into the `colors` dictionary of `vision.py`.

Useful OpenCV and NumPy functions. The following functions may help with the interactive calibration program:

- `cv2.imread(filename)`: Loads the color image in BGR format. Check that loading succeeded before processing the image.
- `cv2.namedWindow(name)` and `cv2.setMouseCallback(name, callback)`: Create an image window and receive mouse events. In the callback, use `cv2.EVENT_LBUTTONDOWN` to add a polygon vertex and `cv2.EVENT_RBUTTONDOWN` to remove the last one.
- `cv2.polylines(image, points, False, color, thickness)`: Draws the selected vertices and edges while the polygon is being defined. Display a copy of the original image so that redrawing does not permanently alter its pixels.
- `cv2.imshow(name, image)` and `cv2.waitKey(delay)`: Display the current selection and read keyboard input. Use the key value returned by `waitKey()` to complete the polygon when `Enter` is pressed.
- `numpy.zeros(image.shape[:2], dtype=numpy.uint8)` and `cv2.fillPoly(mask, [vertices], 255)`: Create a single-channel binary mask and fill the selected polygon.
- `cv2.erode(mask, kernel)`: Shrinks the selected region slightly to exclude uncertain pixels near the object's boundary.
- `cv2.cvtColor(image, cv2.COLOR_BGR2HSV)` and `hsv[mask > 0]`: Convert the image to HSV and extract only pixels inside the eroded polygon. Check that the resulting array is `nonempty`.

- `numpy.median(values, axis=0)` and `numpy.std(values, axis=0)`: Calculate the median and standard deviation of the selected pixels. Apply these directly to saturation and value; handle hue as a circular quantity.
- `numpy.mod(values, 180)`: Helps represent hue on OpenCV's circular range. For example, calculate each hue's signed difference from a chosen center as $(\text{hue} - \text{center} + 90) \% 180 - 90$, then estimate the spread of those differences.
- `numpy.clip(values, lower, upper)`: Restricts saturation and value thresholds to $[0, 255]$. A hue interval that crosses the boundary at $0/179$ must instead be split into two valid intervals.
- `cv2.inRange(hsv, lower, upper)` and `cv2.bitwise_or(mask1, mask2)`: Apply the estimated HSV thresholds to the whole image and combine two masks when a hue interval wraps around.
- `cv2.bitwise_and(image, image, mask=mask)`: Displays only the image pixels accepted by the resulting threshold, making false inclusions and missed object pixels easier to inspect.

Evaluation. Calibrate the thresholds on one image and test them on images of the same object under at least two different lighting conditions. Discuss which pixels are incorrectly included or excluded. Compare the result with the original fixed thresholds in `vision.py` and explain whether the interval median $\pm 2\sigma$ needs a minimum width or manual adjustment for reliable detection.

Structure of `brain.py` and Extension of the Object Interface

The **Brain** node coordinates the user interface, object detection, motion planning, and pump control. Its main components are summarized in Table 3.

Current limitation. The code identifies an object solely by its color. For example, `get_cube_coords(color)` requests one cube, and the collision-object ID is set to the color string. The planning scene always represents an object as a box measuring $0.04 \times 0.04 \times 0.04$ m. Consequently, the current interface cannot distinguish a red cube from a red cylinder or two red objects of the same shape.

Required modifications.

- 1) **Extend the vision service.** Add a `shape` field to the request so that an object can be selected by both color and shape. Replace the special coordinate array used to indicate not detected with an explicit Boolean `found` field. Return a unique object ID together with the pose. If several objects match, specify how one is selected or return the available matches.

- 2) **Adapt the service client.** Replace `get_cube_coords(color)` with a method such as `get_object(color, shape)`. Set both request fields, check `response.found`, and pass the returned ID and coordinates to the manipulation functions. Rebuild the ROS 2 interface package after modifying the service definition.
- 3) **Represent the detected shape in MoveIt.** Replace `spawn_cube()` with `spawn_object()`. Create a `SolidPrimitive.BOX` for a cube and a `SolidPrimitive.CYLINDER` for a cylinder. Use calibrated object dimensions. Identify collision objects by their unique IDs rather than by color; update the attach, detach, and remove operations accordingly.
- 4) **Pass object identity through manipulation.** Update `pick()` and `drop()` to use the selected object's ID and pose. The same ID must be used when the object is added to the planning scene, attached to the pump, detached, and removed. Review `place()` separately: its fixed height increment of 0.04 m assumes a stack of cubes.
- 5) **Extend the terminal menu.** In `choose_pick()`, ask the user to select a color and a shape. If multiple objects match, display their IDs and positions and ask the user to select one before calling `pick()`. Keep the destination selection in `choose_drop()`.
- 6) **Simplify menu control flow.** Replace the repeated calls to `choose_pick()` and `control_menu()` with loops. The command `exit` should leave the current menu without adding another nested function call.

Useful ROS 2 interfaces and Python constructs. The following interfaces and commands may help with the implementation:

- `GetCubeCoords.srv`: Defines the fields exchanged between the vision service and its client. Add `string shape` to the request and fields such as `bool found` and `string object_id` to the response. Retain a pose or coordinate field whose meaning and reference frame are clearly specified.
 - `self.create_service(ServiceType, name, callback)`: Creates the service server in `vision.py`. Adapt its callback to search for objects matching both the requested color and shape.
 - `self.create_client(ServiceType, name)` and `client.call_async(request)`: Create and call the corresponding service client in `brain.py`. Check `response.found` before using the returned pose.
 - `colcon build --packages-select mycobot_interfaces`: Regenerates the Python classes after changing the service definition. Rebuild dependent packages as needed, source the updated workspace, and restart both nodes so they use the same interface version.
-

- `SolidPrimitive.BOX` and `SolidPrimitive.CYLINDER`: Select the collision geometry according to the detected shape. A box uses three side lengths. A cylinder uses `CYLINDER_HEIGHT` and `CYLINDER_RADIUS`; supply a *radius*, not a diameter.
- `CollisionObject.id`: Identifies an object in the planning scene. Assign a unique, sufficiently stable ID to each physical object. Use that same ID for subsequent updates, attachment, detachment, and removal.
- `AttachedCollisionObject`: Represents an object attached to a robot link after grasping. Check that its object ID matches the ID previously added to the planning scene.
- Python dictionaries or data classes: Keep an object's ID, color, shape, dimensions, and pose together when passing it from detection to `pick()` and `drop()`. A dictionary keyed only by color cannot hold two red objects.
- `while` loops with `break` and `return`: Implement repeated menu choices without calling `choose_pick()` or `control_menu()` recursively. Use a single validation step to convert abbreviated user input into a complete color and shape.

If the service is intended to return *all* matching objects, define an appropriate object message and return an array of those messages. A response containing one ID and one pose can represent only one selected match; document the selection rule in that case.

Example interaction.

```
Color: red / yellow / green / blue
Shape: cube / cylinder
Matching objects:
1 - red cylinder at (0.14, 0.03)
2 - red cylinder at (0.20, 0.06)
Select object: 1
Destination bin: A / B / C / D
```

The existing motion-planning methods can largely remain unchanged because they already receive target coordinates. The main change is to preserve the selected object's color, shape, and unique identity from the vision response through the complete pick-and-place sequence.

Assignment: Detecting Object Color and Shape

Extend the vision module so that it detects both the *color* and the *shape* of objects in the camera image. The workspace contains colored cubes and cylinders viewed from above. Cubes should appear approximately square, while cylinders should appear approximately circular.

Existing implementation. In `vision.py`, `img_callback()` creates one HSV mask per color. It finds contours in each mask, filters them by the dimensions and aspect ratio of a minimum-area rectangle, and stores the center of one detected cube per color in `self.detected_cubes`. The service `/cube_coordinates` currently accepts a color and returns the coordinates of the corresponding cube.

Task.

- 1) **Segment objects by color.** Use the HSV ranges to create a mask for each supported color. Apply appropriate mask cleanup before extracting contours. Discard regions that are too small to represent an object.
- 2) **Extract shape features.** For each remaining contour, calculate its area, perimeter, center, minimum-area bounding rectangle, and circularity

$$C = \frac{4\pi A}{P^2},$$

where A is the contour area and P is its perimeter. Approximate the contour with `cv2.approxPolyDP()` and inspect its number of vertices. Explain how these features help distinguish a square from a circle, including the effect of noise and partial occlusion.

- 3) **Classify each object.** Assign a color from the HSV mask and a shape label `cube` or `cylinder` from the contour features. Define and justify classification thresholds using images from the actual camera. Mark uncertain regions as `unknown` rather than forcing a shape label.
- 4) **Store and display detections.** Replace the data structure that stores at most one cube per color with a representation that can hold multiple objects, including objects with the same color. Store at least the color, shape, image center, and orientation when an orientation is meaningful. Draw each contour and its label, such as `red cube` or `blue cylinder`, in the debug window. Remove detections when the objects disappear from the image.
- 5) **Extend the query interface.** Extend the service interface so that a client can request an object by both color and shape. For example, a request for `color = red` and `shape = cylinder` must not return a red cube. Define how the service responds when no matching object exists or several objects match. Update the service definition, server callback, and client code consistently.

Useful OpenCV functions and Python constructs. The following functions may help with color and shape detection:

- `cv2.inRange(hsv, lower, upper)`: Creates a binary mask for one HSV interval. `cv2.bitwise_or()` combines multiple intervals belonging to the same color.
- `cv2.getStructuringElement()` and `cv2.morphologyEx()`: Create a kernel and apply opening or closing to remove isolated pixels or fill small gaps in a color mask.
- `cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`: Finds the outer boundary of each segmented region. Process every plausible contour rather than selecting only one contour per color.
- `cv2.contourArea(contour)`: Returns the enclosed area A in square pixels. Use it to reject regions that are too small or too large.
- `cv2.arcLength(contour, True)`: Returns the perimeter P of a closed contour. Check that $P > 0$ before calculating its circularity $4\pi A/P^2$.
- `cv2.moments(contour)`: Returns image moments. If `m00` is nonzero, the contour center is given by

$$c_x = \frac{m10}{m00}, \quad c_y = \frac{m01}{m00}.$$

- `cv2.minAreaRect(contour)`: Fits a rotated rectangle and returns its center, side lengths, and image angle. The ratio of its side lengths is useful for checking whether a region is approximately square; this ratio alone does not distinguish a square from a circle.
- `cv2.approxPolyDP(contour, epsilon, True)`: Approximates a contour with fewer vertices. Choose `epsilon` relative to the contour perimeter and compare the resulting vertex count with the contour's circularity. A cube viewed from above may produce approximately four corners, whereas a cylinder may produce a near-circular outline.
- `cv2.drawContours()`, `cv2.circle()`, and `cv2.putText()`: Overlay each accepted contour, its center, and its predicted color and shape on the debug image.
- A Python list of detection records: Store one record per object, for example its ID, color, shape, center, image angle, and relevant contour measurements. Create a new list for each processed frame so removed objects do not remain in the current detections.
- A custom ROS 2 `.srv` definition: Add `color` and `shape` to the request. Define whether the response returns one selected match or a list of matches, and include an explicit indication when no object matches. Update the service server and client together.

Use the image angle only when the object's orientation can be determined meaningfully. In particular, an unmarked cylinder with a circular top does not have an observable rotation about its symmetry axis in a top-down image.

Evaluation. Test the implementation with cubes and cylinders of several colors, including two objects of the same color. Show the annotated image and report the detected labels and centers. Test queries for an existing object, an absent object, and multiple matching objects. Discuss at least one case in which the shape classification is uncertain.

Competences

After completing the Pick and Place lab, students will be able to:

- Operate the existing sorting and stacking application and document its behavior and limitations on the real robot.
- Explain how the camera, Vision, Brain, MoveIt, and Controller nodes exchange information through ROS 2 topics, services, and actions.
- Select an object region interactively, analyze its HSV pixel distribution, and derive color thresholds while accounting for the circular hue range.
- Improve color segmentation using morphological filtering and contour checks, and prevent detections from persisting after an object has disappeared.
- Detect two ArUco markers at known locations, estimate the camera extrinsic parameters, and assess the result using the known camera height and marker reprojection errors.
- Transform image measurements into robot-base coordinates using calibrated camera parameters and evaluate the position error at independent workspace locations.
- Extract geometric features from segmented regions and use them to distinguish cubes from cylinders, including cases in which the shape cannot be classified reliably.
- Modify a custom ROS 2 service and its client and server so that objects can be queried by both color and shape and multiple objects of the same type can be handled.
- Represent cubes and cylinders with appropriate collision geometries in the MoveIt planning scene and preserve each object's identity throughout picking, attachment, transport, and release.
- Extend the text-based control menu to select an object by color and shape, and compare the completed pick-and-place behavior with the original implementation.

Component	Responsibility
ROS interfaces in <code>__init__()</code>	Connect to the vision service, MoveIt actions, the Cartesian-path service, collision-object topics, and the pump controller.
<code>get_cube_coords()</code>	Request the coordinates of a cube selected by color.
<code>spawn_cube()</code> , <code>destroy_cube()</code> , <code>attach_cube()</code> , <code>detach_cube()</code>	Manage cubes as collision objects in the MoveIt planning scene.
<code>send_goal_pose()</code> , <code>send_cartesian_path()</code> , <code>go_home()</code>	Plan and execute robot motions.
<code>pick()</code> , <code>place()</code> , <code>drop()</code>	Combine robot motion, pump control, and planning-scene updates into manipulation sequences.
<code>control_menu()</code> , <code>choose_pick()</code> , <code>choose_drop()</code>	Implement the terminal interface for stacking and sorting.

Table 3: Main components of `brain.py`.